

Lecture on Adversarial Search and Types of Games

Course: Artificial Intelligence

Instructor: Syed Hamed Raza

Overview of the Lecture:

This lecture will introduce the concept of **Adversarial Search**, which is a type of search used to make decisions in competitive environments where multiple agents (players) are involved. We will discuss the types of games that can be solved using adversarial search techniques, including **two-player games** and **multi-agent games**. By the end of the lecture, you will have a solid understanding of how adversarial search works, its applications in AI, and how different types of games affect the decision-making process.

Agenda:

1. Introduction to Adversarial Search
 2. Types of Games
 - Two-Player Games
 - Multiplayer Games
 3. Game Tree Representation
 4. Minimax Algorithm for Decision Making
 5. Alpha-Beta Pruning
 6. Classroom Discussion and Problem Solving
-

1. Introduction to Adversarial Search

What is Adversarial Search?

- **Adversarial search** refers to the search process used in environments where there are two or more agents (players) competing against each other, with each agent's actions directly affecting the outcomes of the others. This search technique is common in **game-playing AI**.
- The goal of adversarial search is to determine the best possible move for a player, assuming the opponent also plays optimally. This leads to a process of maximizing one player's utility (the "MAX" player) while minimizing the opponent's utility (the "MIN" player).

Key Characteristics of Adversarial Search:

- **Two or more players:** In these games, players make alternating moves with conflicting goals.
- **Zero-Sum:** Often, adversarial games are **zero-sum** games, meaning that one player's gain is the other player's loss.
- **Perfect Information:** In most games, both players know the entire game state at each move. This is often the case in board games like Chess or Tic-Tac-Toe.

- **Optimal Play:** Adversarial search assumes that both players will play optimally, meaning they will always make the best possible move.
-

2. Types of Games

Two-Player Games

- **Two-player games** are the most common use case for adversarial search. In these games, there are two players: one trying to maximize their own payoff and the other trying to minimize it.
- **Examples:**
 - **Chess:** One player seeks to win by checkmating the opponent, while the opponent tries to do the same.
 - **Tic-Tac-Toe:** A simple example where two players alternate placing their marks on a 3x3 grid.
 - **Connect Four:** A game where two players take turns dropping colored discs into a grid, trying to connect four in a row.

Multiplayer Games

- **Multiplayer games** involve more than two players and require more complex strategies since multiple agents are interacting in the game environment. In these types of games, players might collaborate or compete against multiple opponents simultaneously.
- **Examples:**
 - **Poker:** A game where multiple players compete against each other, trying to outplay each other based on hidden information (cards).
 - **Risk:** A strategy board game where players control armies and aim to conquer territories from other players.
 - **Multiplayer Online Battle Arenas (MOBAs):** Games like **League of Legends** or **Dota 2**, where players are part of teams, but they also compete against other teams.

Imperfect Information Games

- Some adversarial games may have **imperfect information**, meaning players do not have complete knowledge of the game state (e.g., some elements of the game are hidden, like cards in a deck or the moves of opponents).
 - **Examples:**
 - **Poker:** Players do not know the opponent's cards, making it an imperfect information game.
 - **Bridge:** A card game where players don't know their opponents' hands but must make decisions based on what they know.
-

3. Game Tree Representation

A **game tree** is a tree representation of the possible moves in a game. Each node in the game tree represents a game state, and the edges represent the moves taken by the players. The root of the tree represents the current game state, and the leaves represent the terminal states (end states).

- **Branches:** Each move by a player generates a new state, branching out the tree.

- **Max Nodes:** At nodes where the MAX player is to move, the algorithm will choose the move that maximizes their payoff.
- **Min Nodes:** At nodes where the MIN player is to move, the algorithm will choose the move that minimizes the MAX player's payoff.

The game tree can grow exponentially, which is why optimizing search techniques (like Minimax and Alpha-Beta Pruning) is crucial.

4. Minimax Algorithm for Decision Making

The **Minimax Algorithm** is used in adversarial search to determine the optimal move for a player. It operates by creating a game tree, evaluating the terminal nodes, and propagating the values back up the tree to find the best decision.

Steps in the Minimax Algorithm:

1. **Build the game tree** starting from the current state.
 2. **Assign values to terminal states:** Evaluate the utility of each terminal state.
 3. **Propagate the values back to the root:**
 - At **MAX nodes:** Select the maximum value from the child nodes (the best move for the MAX player).
 - At **MIN nodes:** Select the minimum value from the child nodes (the best move for the MIN player).
 4. **Choose the optimal move** for the MAX player based on the evaluation at the root.
-

5. Alpha-Beta Pruning

Alpha-Beta pruning is an optimization technique used to improve the performance of the **Minimax Algorithm** by pruning branches of the game tree that do not need to be explored.

How it Works:

- **Alpha:** The best value found so far along the path of the Maximizer.
 - **Beta:** The best value found so far along the path of the Minimizer.
 - If **Alpha $\geq$ Beta**, there is no need to explore further along this path because the MAX player will not choose this path.
 - This pruning reduces the number of nodes evaluated, making the search faster without affecting the outcome.
-

6. Classroom Discussion and Problem Solving

- **Open Discussion:** Discuss how adversarial search is used in popular games like Chess and Tic-Tac-Toe. What strategies could be employed to deal with imperfect information games?
 - **Hands-on Activity:** Implement the **Minimax Algorithm** for a simple two-player game (e.g., Tic-Tac-Toe) and analyze the results. Try to compare it with and without **Alpha-Beta Pruning**.
-

Conclusion:

- **Summary:** In this lecture, we explored the concept of **Adversarial Search**, its application in **two-player** and **multiplayer** games, and how game trees represent the possible states and moves. We also discussed the **Minimax Algorithm** and its optimization with **Alpha-Beta Pruning** to improve performance in adversarial settings.
- **Next Steps:** In the next lecture, we will dive deeper into **Imperfect Information Games** and explore advanced strategies for these types of games.